

Subject: Data Structure

Time: 1.5 Hours

Semester / Programme: 3rd Semester

Full Marks: 20

Instructions: Answer Question No. 1 (Compulsory) and any THREE from Question No. 2 to 5. Figures in brackets indicate marks.

SECTION - A (Compulsory)

1(a) How many different Binary Search Trees (BST) can be constructed with 4 distinct keys?

[1]

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Solution & Key Concept:

- Use the Catalan number formula for counting BSTs: $C_n = \frac{1}{n+1} \binom{2n}{n}$
- Substitute $n = 4$: $C_4 = \frac{1}{5} \binom{8}{4} = \frac{1}{5} \times \frac{8 \times 7 \times 6 \times 5}{4 \times 3 \times 2 \times 1} = \frac{1}{5} \times 70 = 14$

Answer: 14

Marking Rubric: [1 Mark] for correct formula and final answer

1(b) Write the postfix expression for the following infix expression: $A + B \times C - D / E$

[1]

Solution & Key Concept:

- Apply operator precedence and associativity: multiplication and division have higher precedence.
- Convert sub-expressions: $B \times C \rightarrow BC*$, $D / E \rightarrow DE/$
- Combine with remaining operators: $A + BC* - DE/ \rightarrow ABC*+ DE/-$

Answer: $ABC*+ DE/-$

Marking Rubric: [1 Mark] for correct postfix conversion

1(c) Define a sparse matrix and state its primary storage advantage in computer memory.

[1]

Solution & Key Concept:

- A matrix in which the majority of elements have a value of zero is termed a sparse matrix.
- Primary advantage: By storing only non-zero elements (using triplet representation or linked lists), memory space and execution time are significantly saved.

Answer: A matrix with mostly zero elements; saves memory by storing only non-zero entries.

Marking Rubric: [0.5 Mark] for definition, [0.5 Mark] for storage advantage

1(d) State the worst-case time complexity of searching an element in a Binary Search Tree (BST) of height h .

[1]

$$T(n) = O(h)$$

Solution & Key Concept:

- In the worst-case scenario (e.g., a skewed BST), the tree degenerates into a linear chain of nodes where height $h = n$.
- Searching requires traversing down the single branch, leading to $O(h)$ or $O(n)$ time complexity.

Answer: $O(h)$ (or $O(n)$ for a skewed tree)

Marking Rubric: [1 Mark] for correct time complexity notation

1(e) What is the maximum number of nodes in a binary tree of height h (assuming root is at height 0)?

[1]

$$N_{max} = 2^{h+1} - 1$$

Solution & Key Concept:

- At each level i ($0 \leq i \leq h$), the maximum number of nodes is 2^i .
- Summing the nodes across all levels from 0 to h : $\sum_{i=0}^h 2^i = 2^{h+1} - 1$.

Answer: $2^{h+1} - 1$

Marking Rubric: [1 Mark] for correct formula

SECTION - B (Answer any THREE)

Answer any THREE questions from Q.2 to Q.5. Each question carries 5 marks.

Q.2. (a) Write a C/C++ function to implement PUSH and POP operations on a Stack represented using a Singly Linked List. [3]

Solution for (a):

- Define Node structure with data and next pointer.
- For PUSH(x): Create a new node, set its data to x, point its next to top, and update top = new_node.
- For POP(): Check underflow (top == NULL), retrieve data from top, update top = top->next, and free the old top node.

Result / Conclusion: Implementation of push and pop using dynamic memory allocation.

Marking Rubric: [1.5 Marks] for PUSH logic, [1.5 Marks] for POP logic with underflow check

(b) Explain dynamic storage management and how garbage collection helps in memory reclamation. [2]

Solution for (b):

- Dynamic storage management refers to the allocation and deallocation of memory from the heap during runtime.
- Garbage collection automatically identifies and reclaims memory occupied by objects that are no longer accessible or referenced by the program, preventing memory leaks.

Result / Conclusion: Concept of heap allocation and automated reclamation of unreachable memory blocks.

Marking Rubric: [1 Mark] for dynamic storage definition, [1 Mark] for garbage collection role

- Q.3. (a) Construct a binary tree given the following traversal sequences:\nPreorder: A B D E C F\nInorder: D B E A F C [3]

Solution for (a):

- Identify root from Preorder: 'A' is the first element, so 'A' is the root.
- Locate 'A' in Inorder to divide left and right subtrees: Left Inorder = [D, B, E], Right Inorder = [F, C].
- Corresponding Preorder subsets: Left Preorder = [B, D, E], Right Preorder = [C, F].
- Recursively build left subtree (Root 'B', left child 'D', right child 'E') and right subtree (Root 'C', left child 'F').

Result / Conclusion: Binary tree with root A, left subtree rooted at B (children D, E), and right subtree rooted at C (left child F).

Marking Rubric: [1.5 Marks] for correct identification of root and subtree divisions, [1.5 Marks] for final tree structure

- (b) Write a recursive function to find the height of a given binary tree. [2]

$$\text{Height}(T) = 1 + \max(\text{Height}(\text{left}), \text{Height}(\text{right}))$$

Solution for (b):

- Base case: If the tree/node is NULL, return -1 (or 0 depending on convention).
- Recursive step: Compute height of left subtree and right subtree.
- Return $1 + \max(\text{height}(\text{left}), \text{height}(\text{right}))$.

Result / Conclusion: `int height(Node* root) { if(root == NULL) return -1; return 1 + max(height(root->left), height(root->right)); }`

Marking Rubric: [1 Mark] for base case, [1 Mark] for recursive logic and max calculation

- Q.4. (a) Perform operations on polynomials: Represent polynomial $SP_1 = 4x^3 + 3x^2 + 5$ using a singly linked list and write an algorithm to add two polynomials. [3]

Solution for (a):

- Node structure representation: Each node contains `coefficient`, `exponent`, and `next` pointer.
- Represent SP_1 : Node(4,3) -> Node(3,2) -> Node(5,0) -> NULL.
- Addition Algorithm: Initialize pointers `p1` and `p2` for both polynomials. Compare exponents: if equal, add coefficients and advance both; if $p1 \rightarrow \text{exp} > p2 \rightarrow \text{exp}$, add `p1` term and advance `p1`; else add `p2` term and advance `p2`.

Result / Conclusion: Algorithm and linked list node structure for polynomial addition.

Marking Rubric: [1 Mark] for node structure and representation, [2 Marks] for addition algorithm logic

- (b) Explain the advantages of Circular Linked Lists over Singly Linked Lists with a suitable example. [2]

Solution for (b):

- In a circular linked list, the last node points back to the first node, forming a circle.
- Advantage 1: Any node can be reached from any other node without traversing from the head pointer.
- Advantage 2: Particularly useful for applications like round-robin CPU scheduling and circular queues where continuous traversal is required.

Result / Conclusion: Enables continuous traversal from any node and simplifies queue implementations.

Marking Rubric: [1 Mark] for structural explanation, [1 Mark] for clear advantages

- Q.5. (a)** Write a non-recursive pseudo-code/algorithm to traverse a Binary Tree in In-order using an explicit stack. [3]

Solution for (a):

- Initialize a stack and set `curr = root`.
- While `curr != NULL` or stack is not empty:
- While `curr != NULL`, push `curr` to stack and move `curr = curr->left`.
- Pop top node from stack, visit it (`print node->data`), and set `curr = node->right`.

Result / Conclusion: In-order iterative traversal algorithm using Stack.

Marking Rubric: [1.5 Marks] for left-ward traversal and pushing to stack, [1.5 Marks] for popping, visiting, and shifting to right subtree

- (b)** Convert the following infix expression into its prefix equivalent: $(A + B) \times C - (D / E)$ [2]

Solution for (b):

- Reverse the infix expression and swap parentheses: `) E / D ( - C * ) B + A (`
- Convert reversed infix to postfix:
- - `(E D /)` -> `ED/`
- - `(B A +)` -> `BA+`
- - Combine with operators: `ED/ C * BA+ -`
- Reverse the resulting postfix expression to get the prefix expression: `- * C + A B / D E`

Result / Conclusion: `-*C+AB/DE`

Marking Rubric: [1 Mark] for reversal/preprocessing steps, [1 Mark] for correct prefix output

--- END OF PAPER ---